

Backend: Kubernetes Deployment Guide

Welcome to the Kubernetes deployment configuration for the **Backend**. This folder (`k8s/backend/`) contains the YAML files that instruct Kubernetes on how to run our Spring Boot Java API and connect it to the database.

Let's dissect these files to understand how they work together.

1. `deployment.yml` (The Application Pods)

Purpose: To pull our Spring Boot Docker image, run it inside a pod, and inject the environment variables it needs to find the database.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: portfolio-backend
```

- **What is a Deployment?** The "manager" resource in Kubernetes. It oversees the pods. If our Java application crashes due to an out-of-memory error, this Deployment will immediately create a fresh pod to replace it.

```
- name: backend
  image: srtechops/three-tier-k8s-project:backend-latest
  ports:
    - containerPort: 8080
```

- **Container Specifications:** We declare our image repository and tag. We also expose port `8080` , which is the default port for embedded Tomcat in Spring Boot.

The Magic of Environment Variables

```
env:
  - name: DB_HOST
    value: portfolio-mysql-0.mysql
  - name: DB_USER
    value: root
```

- **Service Discovery:** How does the Backend know where the Database is? We use Kubernetes Internal DNS! Kubernetes automatically resolves `portfolio-mysql-0.mysql` to the running MySQL pod.

- **Overrides:** Inside the Java code (`application.yml`), the database URL is looking for a `${DB_HOST}` variable. By defining it here in the deployment YAML, we override the default `localhost` setting without changing a single line of Java code!

```
- name: DB_PASS
  valueFrom:
    secretKeyRef:
      name: mysql-secret
      key: mysql-root-password
```

- **Security Best Practice:** We never type raw passwords into deployment files. Instead, we reference the encrypted K8s Secret created for the database (`mysql-secret`) and extract precisely the key we need. This means only Kubernetes memory knows the true password.

2. `service.yml` (The Cluster Network Exposer)

Purpose: To give our dynamic backend pods a stable network name and load balance incoming requests from the Frontend.

```
apiVersion: v1
kind: Service
metadata:
  name: portfolio-backend-service
```

- **Crucial Naming:** Remember the Frontend's `config.yml` ? It forwards API calls to `http://portfolio-backend-service:8080` . That magic routing works strictly because the K8s internal DNS uses the `metadata.name` of this service as a hostname!

```
spec:
  type: NodePort
  selector:
    app: backend
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
      nodePort: 30081
```

- **Selector (`app: backend`):** This is the glue. The deployment labeled its pods with `app: backend` , and the Service uses this selector to find them and route network traffic to them.

- **NodePort:** While `ClusterIP` is normally enough for a backend (since only the frontend needs to talk to it), exposing it as a `NodePort` (30081) is great for us as developers. It allows us to use tools like Postman on our host laptops to test the API directly without going through the frontend proxy!
-

Summary for Students

The beauty of Kubernetes is **Decoupling**. Our Java application has no idea it is running in K8s, and has no idea what the database IP address is.

Kubernetes uses the `Deployment` to inject that intelligence dynamically at runtime via `env` variables, and uses the `Service` to give the backend a permanent name that the frontend can rely on.